

nter notice or remove the content from gitlab.com

- GitLab users, implicated by the notice, have the right to submit a counter-notice. If the counter-notice is deemed valid, we will forward it to you for further action.
- For more information on counter complaints, see the Responding to a DMCA notice section on this page.

{{% /details %}}

Responding to a DMCA Notice

{{% details summary="Submitting a DMCA Counter Notice" %}}

If it is not possible to come to an agreement with the reporting copyright owner, the content owner may submit a DMCA Counter-Notice to dmca@gitlab.com within ten (10) business days of the date of receiving our notice.

If we receive a valid counter-complaint, we will email a copy of the notice to the copyright owner.

The Counter complaint is a legal document and must comply with the requirements of the DMCA and must include the following:

1. Contact Information: Your name, title (if acting as an agent), physical address, email address, and phone number;
1. Content Identification: A reasonable identification of the material that has been removed or to which access has been restricted;
1. Statement under Perjury: A statement under penalty of perjury that you have "a good faith belief that the material was removed or disabled as a result of mistake or misidentification of the material to be removed or disabled.";
1. Consent to Jurisdiction: A statement that you consent to the jurisdiction of Federal District Court for the judicial district in which your provided address is located, or if your address is outside of the United States, for any judicial district in which GitLab may be found.
1. Consent to service of process: A statement that you will accept service of process from the person who provided the original notification or an agent of such person.
1. Signature: An electronic or physical signature of the owner of the copyright or a person authorized to act on the owner's behalf.

{{% /details %}}

{{% details sunmmary="Mitigating Copyright Infringement" %}}

- Should you receive a gitlab.com DMCA notice for content that is in your control;
 1. You can respond to the notice by removing the reported content from gitlab.com.
 1. If you believe your content is mistakenly the subject of a DMCA take down request, you have the right to contest the take down request by submitting a counter complaint.
- In the event that we do not see that any actions has been taken and no response to the DMCA notice has been received;
 1. The reported content will be made private.
 - Depending on where the content is located, access to the group/project can be restricted.
 - Access to the gitlab.com account is restricted.

{{% /details %}}

Trademark Infringement

{{% details summary="Submitting a Trademark Infringement Notice" %}}

To submit a trademark infringement report under the Trademark Policy, please provide:

1. Evidence that you are holder of a relevant trademark registration, such as a scanned copy of the trademark registration certificate(s) or screenshot of the registration on the website or database of the applicable national intellectual property office(s).

1. A declaration of the following statement: "By submitting this trademark infringement notice, I state that I have a good-faith belief that the reported use, in the manner that I have complained of, is not authorized by the intellectual property rights owner, its agent or the law; that the information contained in this notice is accurate; and, under penalty of perjury, that I am authorized to act on behalf of the owner of the intellectual property rights at issue".

You can send this information to dmca@gitlab.com, you should receive an automated confirmation of receipt.

{{% /details %}}

Name Squatting

{{% details summary="Requesting a dormant namespace" %}}

The GitLab.com Support Team will consider a namespace (user name or group name) to fall under the provisions of this policy when the user has not logged in or otherwise used the namespace for an extended time.

Specifically:

User namespaces can be reassigned if both of the following are true:

- The user's last sign in was at least two years ago.
- The user is not the sole owner of any active projects.

Group namespaces can be reassigned if one of the following is true:

- There is no data (no project or project(s) are empty).
- The owner's last sign in was at least two years ago.
- If the namespace contains data, GitLab Support will attempt to contact the owner over a two-week period before reassigning the namespaces. If the namespace contains no data (empty or no projects) and the owner is inactive, the namespace will be released immediately.
- Namespaces associated with unconfirmed accounts over 90 days old are eligible for immediate release. Group namespaces that contain no data and were created more than 6 months ago are likewise eligible for immediate release.

Account and Group ownership disputes should contact GitLab Support to see if namespaces are eligible.

{{% /details %}}

SECTION: security/standards/_index.md

List of active internal standards

Please visit the internal security standards

List of active public standards

SECTION: security/standards/cryptographic-standard.md

Purpose

The Cryptography Standard defines approved cryptographic algorithms, settings, and cryptographic modules for the purposes of encrypting data at rest or in transit within the various systems and subsystems used by the GitLab product.

The Cryptography Standard allows for a more consistent approach to cryptographic usage within GitLab, easier adaptation to industry standards and compliance frameworks (such as FedRAMP), and an overall more secure product and working environment. In addition, most of the standards are based off of recommendations from NIST (National Institute of Standards and Technology) as many of the compliance frameworks are based off of NIST standards, and NIST consistently makes solid recommendations that are adopted by many organizations globally.

Scope

The Cryptography Standard applies to all GitLab team members, contractors, consultants, vendors and other service providers that handle, manage, store or transmit GitLab data.

This is required for coding best practices as well as client and server configurations involving cryptography for the GitLab product itself. Currently, there are numerous efforts internally by numerous Engineering teams ensuring these standards are met and maintained. The scope includes third party modules and software settings where cryptographic settings are required. Basically if it touches GitLab or GitLab customer data, the standard applies.

Roles & Responsibilities

Role	Responsibility
GitLab Team Members	Responsible for adhering to the requirements outlined in this standard
Security Management and Cryptographic Officer (Code Owners)	Responsible for approving significant changes and exceptions to this standard

GitLab Responsibilities

- GitLab team members, contractors, consultants, vendors and other service providers are required to review and understand this cryptographic standard, and how to handle encryption needs for data at rest and in transit as defined below unless otherwise noted.

Customer Responsibilities

- GitLab customers are responsible for managing their own data, should consider these standards as highly recommended, and adopt them according to their own internal requirements. GitLab handles Customer Data internally according to our non-disclosure obligations written in our Mutual Non Disclosure Agreement, and we use the controls identified in this standard.

Standard

Compliance and Certification Standards

While we consider these standards to be a security baseline to improve the overall security of the GitLab product, for compliance and certification efforts we will be using the following general guidelines.

- No non-public cryptographic modules. All cryptographic modules should be open source and well maintained. This is to ensure compliance with the Bureau of Industry (BIS) and Security's Export Administration Regulations regarding encryption.
- All product development shall include support for GitLab's FIPS releases, following the patterns specified in the FIPS development documentation on the roadmap. New features that are not leveraging underlying FIPS-validated cryptographic modules cannot be made available to customers who use our FIPS releases or Dedicated for Government offering. They should be disabled by default behind a feature flag or, if not possible, an option to disable the feature entirely.
- All cryptographic choices (algorithms, etc) will adhere to FIPS 140-3 minimum standards (which superseded FIPS 140-2 on 03/22/2019), and keep in mind NIST SP 800-53 as well.
- When possible, choose quantum-resistant algorithms as an effort to help "future proof" cryptographic choices as quantum computing usage becomes more powerful and more mainstream. For more information read the section on quantum cryptography [here](#).

Meeting these standards will help meet objectives such as FedRAMP compliance, but as standards such as FIPS and SP 800-53 are considered de facto global standards, it will help with compliance needs in multiple countries and industries.

Cryptographic Modules

For regulatory requirements (such as FIPS), all cryptographic modules must be publicly available (open source) to ensure compliance with the Bureau of Industry and Security's Export Administration Regulations regarding encryption. As this compliance meets international standards per the Wassenaar Arrangement this should meet any requirements that might be encountered globally, should the situation arise.

The choice of cryptographic modules is important. It is possible that GitLab features that have cryptographic elements could find themselves having to meet stringent module requirements (FedRAMP standards, running in FIPS mode, etc) so adherence to specific modules is highly recommended. This includes the following:

- OpenSSL Cryptographic Module
- Linux Kernel Crypto API Cryptographic Module
- Libgcrypt Cryptographic Module

For the gitlab-fips package, all FIPS-validated cryptographic modules are documented on the "FIPS Compliance" Docs page, currently located [here](#). This documentation must be kept up to date to maintain GitLab's FedRAMP and FIPS 140-2 compliance posture.

Note to developers and contributors, regardless of FIPS/FedRAMP compliance or not, coding should use the cryptographic modules mentioned above. Development with a non-FIPS version of the same module (e.g. openssl-x.y.x vs openssl-x.y.z-fips) is fine. The important point is to not introduce new code that is dependent on cryptographic modules that do not have a FIPS version or have written their own cryptographic implementations of algorithms. A list of FIPS-certified cryptographic modules can be found [here](#).

Algorithmic Standards

- TLS - TLS versions 1.2 or 1.3 are required. TLS 1.0 and 1.1 are not allowed, despite being FIPS 140-2 compliant. FIPS 140-3 and SP 800-53 standards disallow TLS 1.0 and 1.1 explicitly, as does FedRAMP which follows those same guidelines.

- Here are the allowed algorithms in order of preference:

- ECDHE-ECDSA-AES256-GCM-SHA384
 - ECDHE-RSA-AES256-GCM-SHA384
 - ECDHE-ECDSA-AES128-GCM-SHA256
 - ECDHE-RSA-AES128-GCM-SHA256
 - TLSAES256GCM SHA384
 - TLSAES128GCM SHA256

- In regards to coding practices, refer to the secure coding guidelines, in particular for language-specific guidelines as well as limitations.

- ML-KEM is a NIST-approved quantum safe algorithm. Specifically ML-KEM-768 is recommended. Just note that ML-KEM can only be used in a FIPS environment if it is using a FIPS-certified cryptographic module. For more information see this section.

- Block Ciphers - AES. Currently AES-128 is considered the minimal, but to ensure "future proofing" AES-256 is preferred (note there are potential performance impacts depending upon application usage, etc). GCM is the required mode (e.g. aes-256-gcm).

- It should be noted that AES-256 is considered quantum resistant, which is also another consideration for future proofing.

- NOTE for Laptops: For full-disk encryption XTS-AES-128 (with 256 bit key) or XTS-AES-256 (with 512 bit key) is acceptable. For both GitLab-approved laptop scenarios for team members (Apple running MacOS or a Linux-based laptop running Ubuntu) one of these two algorithms is automatically pre-chosen and is acceptable when setting up full disk encryption on a laptop.

- Digital Signatures - RSA), ECDSA, and EdDSA. For RSA, the minimum key size is 2048. For ECDSA the minimum key size is 224. Ed25519 and Ed448 are now both approved algorithms if the underlying crypto module has included it in its certification, however at this time our OpenSSL implementation does not certify it.

- ML-DSA is a NIST-approved quantum safe algorithm. Specifically ML-DSA-65 with a random seed of 256 bits is recommended. Just note that ML-DSA can only be used in a FIPS environment if it is using a FIPS-certified cryptographic module. For more information see this section.

- SLH-DSA is a NIST-approved quantum safe algorithm. SLH-DSA's strength lies in functioning as a replacement of digital signatures based only on known-strong hash function design. It replaces stateful hash-based signatures such as LMS and XMSS with a probabilistic construction and does not rely on math such as discrete log or lattices. Just note that SLH-DSA can only be used in a FIPS environment if it is using a FIPS-certified cryptographic module. For more information see this section. Generally ML-DSA is preferred to SLH-DSA except for applications

with a long time-horizon or tight hardware constraints (such as embedded contexts).

- Hash Functions - The SHA-2 family of hash algorithm, with SHA-256 as the minimum. Ideally SHA-384 or higher would be better, although there might be less support particularly in third party software.
 - It should be noted that while SHA-1 is FIPS 140-2 compliant, per NIST SP 800-53 Rev 5 Baselines it is not allowed. This is per SP-800-57 section 5.6.1.2, SP-800-131 section 9 table 8, and SP-800-107 section 4.
 - It should be noted that SHA-2-256+ and SHA-3-256+ are considered quantum resistant.
- MAC - MAC (Message Authentication Code) is usually implemented to authenticate messages as unaltered after transmission. A MAC is to only be used in conjunction with an approved algorithm, and with the following parameters:
 - HMAC, KMAC. Only with a key length equal to or greater than 112 bits.
 - CMAC, GMAC. Only in conjunction with AES; HMAC is generally preferred to CMAC. GMAC should not be used for general purposes.
- Random Number Generation - True random number generation is not required, but a high level of randomization is still important. The preference is to use pseudo-random number generation using /dev/random. While /dev/urandom is not as secure, historically it performed quicker on much older systems. When the underlying operating system is running in FIPS mode, calls to /dev/urandom are directed to /dev/random, and starting with Linux kernel version 5.18 both /dev/random and /dev/urandom will function the same anyway. Therefore the standard is /dev/random.
- Password Hashes - Currently the GitLab product uses the popular Bcrypt algorithm for user account password hashing by default. Bcrypt is not an approved FIPS 140-2 algorithm, so development engineers have added support for PBKDF2+SHA512 starting with GitLab version 15.2 via feature flags, and starting with 15.6 it is available when FIPS-mode is enabled. More details are available [here](#). Key stretching is used for both algorithms, with Bcrypt's factor set to 10 (there are plans to increase this) and PBKDF2+SHA512's factor set to 20,000. Passwords should be salted with a unique salt using one of the approved cryptographic one-way hashes above before Bcrypt or PBKDF2+SHA512 are used, as this will further strengthen the password hash against brute force and dictionary style attacks.
- Note on Older/Alternate Algorithms - Notes on a few other popular algorithms:
 - DSA is no longer an approved algorithm per FIPS 186-5 and is largely regarded as a weak cipher.
 - SHA-1 - For hashing algorithms, SHA-1 is allowed for non-cryptographic functions (such as a checksum). It is not allowed for digital signature generation. For digital signature validation, SHA-1 is allowed. For non-digital-signature applications, SHA-1 is allowed if collision resistance is not required, such as an initialization vector of a block cipher that doesn't require randomization. For more details on SHA-1 usage, refer to NIST SP 800-131A Rev 3, especially section 9 on key derivation methods. As SHA-1 will eventually no longer be allowed in various standards in the future, use SHA-256 whenever possible.
 - MD5 - Previously, MD5 was allowed for non-cryptographic functions. However as a result of initial audit findings from May 2, 2022 MD5 is no longer allowed, whether the function is cryptographic or not. For FIPS compliance MD5 cannot be used at all.

Quantum Cryptography

Quantum computing allows for advanced mathematical calculations, and as quantum-based computers evolve and become more powerful, there is a danger of certain algorithms being able to be broken or "cracked" - namely a number of algorithms that use a form of factoring large numbers as a part of the cracking effort. So there is concern that when this point is reached, many

cryptographic algorithms will no longer be considered secure. This scenario is often referred to as "Post Quantum Cryptography", or PQC for short. This differs from Quantum Cryptography: the former runs on present computing hardware while the latter requires specialized quantum-aware hardware (sometimes also referred to as quantum key distribution (QKD)). Use of Quantum Cryptography is not recommended as part of this document.

In preparation for adopting the new NIST PQC Algorithms, GitLab will consider the following:

- Where possible, choose existing cryptographic algorithms that are already known to be resistant to cracking via quantum computing, commonly known as "quantum resistant".
- As new quantum resistant algorithms are finalized into NIST standards, the standards listed above will be updated.
- GitLab will begin testing these algorithms (ML-DSA, SLH-DSA, and ML-KEM) to ensure that we will be able to migrate to their implementation as more adoption occurs.
- As of August 13, 2024 NIST published the final versions of FIPS 203, FIPS 204, and FIPS 205. Currently the quantum safe algorithms referenced in these standards are approved for use. Ensure that any use of these algorithms in a FIPS environment are using FIPS certified cryptographic modules. Quantum safe cryptographic modules do exist (such as liboqs and OpenSSL 3.5.0), but as of April 2025 none of the existing quantum safe cryptographic modules are FIPS certified.

Exceptions

Exceptions to this policy will be tracked as per the Information Security Policy Exception Management Process.

References

- Controlled Document Procedure

SECTION: security/standards/data-classification-standard.md

Purpose

The Data Classification Standard defines data type and categories and provides the associated Data Classification of each for the purposes of determining the level of protection to be applied to GitLab and Customer data throughout its lifecycle.

Scope

The Data Classification Standard applies to all GitLab team members, contractors, consultants, vendors and other service providers that handle, manage, store or transmit GitLab data.

Roles & Responsibilities

Role	Responsibility
------	----------------

-----	-----
-------	-------

GitLab Team Members	Responsible for adhering to the requirements outlined in this standard
---------------------	--

|
| Data Owners | Responsible for approving exceptions to this standard for their owned data types. These are generally the Business Owners of a system. |
| Security and Legal (Code Owners) | Responsible for approving significant changes and exceptions to this standard |

GitLab Responsibilities

- GitLab team members, contractors, consultants, vendors and all other service providers acting on behalf of GitLab are required to review and understand this data classification standard, and how to handle data according to the classification levels below unless otherwise noted.
- Data Owners shall determine the classification of data in accordance with this standard. The Data Classification Index (internal only) provides a list of various types of data and their classification level. If you cannot identify the data element or are uncertain of the risk associated with the data and how it should be classified and handled, please contact the Security Risk team in Slack via @security-risk.
- To maintain our culture of security, transparency and to minimize the risk to our sensitive data and our customers, GitLab team members are required to complete Data Classification Training as part of GitLab's Security Awareness Training to help understand the different types of data at GitLab and how to keep it SAFE. Training is available via Level Up, GitLab's internal learning platform.

Customer Responsibilities

- GitLab customers are responsible for managing their own data, to include identification and classification according to their own internal requirements. GitLab handles our Customer's Data internally according to our non-disclosure obligations written in our Mutual Non Disclosure Agreement and the classifications identified in this standard.

Standard

Data Classification Definitions

- Personal Data: Any data, individually or when combined with other data, that identifies, relates to, describes or is reasonably capable of being associated with or linked to an identifiable natural person (a 'data subject'), whether directly or indirectly.
- Customer's Data: Refers to the electronic data, originating from the GitLab platform and supporting infrastructure, that was uploaded/created/generated by GitLab customers and processed in the GitLab application with a label of Private, Confidential, or Internal by the customer and subject to legal or contractual obligations. Note that Customer's Data (RED Data) does not include data elements about the customer such as contact or billing information (Orange Data).

{{% alert title="Sharing of Customer-provided data within GitLab" %}}

Regardless of whether customer uploaded/created/generated data exists within private (e.g., projects, groups, sub-groups, and profiles), confidential (e.g., issues and epics), or internal (e.g., comments/notes) GitLab objects, it should not be shared with third parties unless they

are an approved sub-processor, they are sharing data for required legal compliance purposes, or a separate legal approval has been obtained.

{{% /alert %}}

Data Classification Levels

Examples of each data type: See Data Classification Index (internal only)

RED{.text-danger red}

Restricted and must remain confidential. This is GitLab's most sensitive data and access to it should be considered privileged and must be explicitly approved. Exposure of this data to unauthorized parties could cause extreme loss to GitLab and/or its customers. In the gravest scenario, exposure of this data could trigger or cause a business extinction event.

Examples include:

- Customer Data (see definition above in the Data Classification Definitions section)

Red Data may not be transmitted from an approved Red data source to any other systems or solutions without first obtaining approval from the Privacy and Security teams. Any Vendors that process Red Data must first undergo a factual and legal analysis that justifies their processing in accordance with our Customer agreements, as well as global privacy and data security laws. For any questions or concerns related to the transmission of Red data between systems, please reach out to @Security-Risk within the Sec-Assurance channel.

<!-- markdownlint-disable MD022 -->

ORANGE

{style="color: fd7e14" orange}

<!-- markdownlint-enable MD022 -->

Data subject to laws and regulation that should not be made generally available. Unauthorized access or disclosure could cause significant or financial material loss, risk of harm to GitLab if exposed to unauthorized parties, break contractual obligations, and/or adversely impact GitLab, its partners, employees, contractors, and customers.

Examples include:

- Personal Data

- Any vendor who is in possession of any form of Personal Data must have appropriate contractual terms that address GitLab data protection requirements (e.g. a Data Processing Agreement).

- If Personal Data comprises a part of the data set to be processed, then the data classification for that data set should be Orange and the classification cannot be Yellow or Green, even if the majority of the data set is Yellow or Green data.

- The source of the Personal Data should not change its classification to a level below Orange since Personal data gathered from public sources is not exempt from protection under certain data protection laws.

- If you have doubts as to whether something is Personal Data, please see an exhaustive list of Personal Data elements in the Data Classification Index (internal only)

- GitLab Intellectual property
- Customer metadata
- Audit logs
- Open security incidents, vulnerabilities and risks

{{% alert title="Personal Data Exception" %}}

While Personal Data is classified as Orange, there is an exception for GitLab Team Member names, their work email addresses, and their GitLab usernames, which are classified as Yellow. These two Personal Data elements are not considered high-risk or sensitive types of Personal Data. Given GitLab's value of transparency and because GitLab is public by default, most Team Member names are available publicly. As they are often processed in support of everyday corporate operations, the application of Orange-level controls for these lower risk data elements would disproportionately inhibit GitLab's business functions.

{{% /alert %}}

{{% alert title="Personal Data and Team Member Safety" %}}

Please be aware of how combining data elements could lead to impacting a Team Member's safety. For example, Team Member names are classified as Yellow, per the classification description below. But if you combine a Team Member's name along with their dates of travel and site for a work event, then you are possibly revealing that Team Member's exact location, which is Orange level Personal Data. Any document or Issue containing a Team Member's specific location should be set as "Confidential" in accordance with our Orange Data and Confidentiality Levels guidelines.

{{% /alert %}}

<!-- markdownlint-disable MD022 -->

YELLOW

{style="color: ffc107" yellow}

<!-- markdownlint-enable MD022 -->

Data and information that should not be made publicly available that is created and used in the normal course of business. Unauthorized access or disclosure could cause minimal risk or harm and/or adversely impact GitLab, its partners, employees, contractors, and customers.

Examples include:

- Asset registers
- General internal company communications
- Vendor contracts
- GitLab runbooks/work instructions/manuals/policies/procedures containing data NOT appropriate for public consumption
- GitLab Team Member names

GREEN{.text-success green}

Data that is publicly shareable, and does not expose GitLab or its customers to any harm or material impact.

Examples include:

- GitLab handbook
 - Including most GitLab runbooks/work instructions/manuals/policies/procedures
- Public announcements
- Public product information

Data Classification Standards

Credentials and access tokens are classified at the same level as the data they protect

Credentials such as passwords, personal access tokens, encryption keys, and session cookies derive their classification from the highest classification of the data they protect.

Combinations of data types may result in a higher system classification level

If there is more than one data type residing in a system, the system should be classified at the highest data classification level of the data being stored, transmitted or processed on that system.

Labeling

There is currently no internal requirement to label data according to this standard, however labels are encouraged. By labeling data according to classification level, individuals can quickly refer to this policy for proper handling.

Exceptions

Exceptions to this policy will be tracked as per the Information Security Policy Exception Management Process.

References

- Controlled Document Procedure
- Data Classification Index (internal only)

SECTION: security/standards/gitlab_projects_baseline_requirements.md

Purpose

The purpose of this page is to outline a minimum set of requirements that should be adhered to when configuring projects at GitLab that ultimately impact the code which makes up the GitLab product and/or which impact the build, release or deployment of GitLab. The end goal being that no changes that ultimately impact the GitLab codebase can be made without review. This page can also help users of GitLab by offering some possible configurations for securing projects they deem critical.

Given GitLab's commitment to dogfooding and keeping in mind GitLab's Efficiency Value it is important to understand that these baseline project configurations are not required on all projects, only those projects which impact the GitLab codebase (directly or indirectly) or are

otherwise deemed critical for another communicated reason (note in these instances the rationale of the "why" behind the need for these baseline configurations should be able to be clearly communicated).

That said, it is encouraged for team members to keep these baseline configurations in mind when creating or working in projects as they are designed to improve the security and quality of contributions made in GitLab projects.

Scope

The scope of the projects which should have these baseline configurations applied are any project which influences the GitLab codebase directly or indirectly and/or which impact the build, release or deployment of GitLab.

Examples of a project that fits this criteria:

- A project that builds a component of GitLab
- A project that builds a bot which helps manage the codebase for a component of GitLab
- A project used for updating a critical non-GitLab system's codebase (e.g. Salesforce)

Examples of a project that do not fit this criteria:

- A project that has no code
- A project that builds a bot which doesn't impact the GitLab codebase (e.g. a script to convert a YAML file into a CSV)

Project usage may naturally shift over time and it's important to be re-evaluating project usage on an ongoing basis, projects may go in and out of either category so please keep that in mind as you work in projects each day. The hope is that even when a project does have these baseline configurations setup, efficiency isn't decreased as we have team members collaborating on work and avoid working in silos of knowledge that increase our risk of an individual becoming a single point of failure as well. These values also encourage our Transparency Value by getting multiple team members involved in changes.

I'm not sure where my project fits

That's okay! And certainly not going to be uncommon. If you have questions, please raise them so we can address and try to fit into the appropriate category. We can iterate here on how we categorize projects so we ensure our commitment to Security and Quality, as well as Efficiency.

My project doesn't fit seem to fit the criteria but I still need these configurations?

There also may be instances when a project doesn't strictly meet our general criteria for requiring baseline configurations but for some reason it is determined that the project should be configured using the baseline configurations. In these instances the rationale of the "why" behind the need for these baseline configurations should be able to be clearly communicated. If many similar rationales are being provided for projects that don't appear to meet the existing criteria, it is likely an indicator that the scope and criteria of projects should be adjusted.

Protected Branch Baseline Configurations

As a general rule, we want to protect our branches in such a way that we require everyone to submit MRs for that protected branch. This requirement to use an MR allows for an easier trail to follow of changes made and rationale behind them, and most importantly, it requires an MR to be used to make changes to the protected branch which works hand-in-hand with our configured MR Approval Rules. If accounts can push direct to the protected branch, that account does not have to use an MR and can make changes without another team member's involvement.

Example 1 of Protected Branch Settings configured to Require an MR

See Note on usage of Code Owners

MR Approval Rule Configurations

MRs should be reviewed following GitLab's Code Review Guidelines. To ensure MRs are not merged with unreviewed commits, MR Approval Rules should be enabled in one of the two below methods.

Example 1 of MR Approval Rules configured WITHOUT Code Owners

Example 2 of MR Approval Rules configured WITH Code Owners

See Note on usage of Code Owners

Note on usage of Code Owners

Note that not all projects will have Code Owners enabled as it may not be necessary in all instances.

If a project does use Code Owners, the Protected Branch settings should be configured to have "Code owner approval" toggled ON and an appropriately configured Code Owners file should be created. The GitLab Docs defines how Code Owners work and how to set up Code Owners.

The MR Approval Rules should also be configured to utilize Code Owners and require at least one Code Owner to approve MRs with affected file types

Please review thoroughly and ask questions in the sec-assurance slack channel, or tag in the MR you're setting up Code Owners with if there is any confusion!

Ongoing Monitoring

Please note that projects that meet the criteria for requiring these baseline configurations may be selected at any point for testing of configurations by the GitLab Security Compliance team as part of our continuous control monitoring program to make sure we're adhering to the guidance outlined on this page. Please see the GCF Security Control Lifecycle page for an overview of the program.

The Security Compliance team has also implemented live monitoring using detections/alerts in the Devo SIEM tool and Slack for all MR Approval and Protected Branch settings outlined above. If MRs were merged without proper approval/SOD in place as a result of the changed settings an observation issue will be opened to identify the root cause and perform remediation. Remediation will include steps such as gaining an understanding of why they were changed, revert the settings back to being in compliance, and reviewing any MRs that were merged without proper approval or segregation of duties (SOD) in place to ensure no malicious changes were included obtaining retroactive approvals if needed. Security Compliance will also loop in the individual(s) who made the change as well as the management of the team that owns/works on the project in order to help identify the root cause of the changes and to work towards implementing workflow changes so that the settings do not need to be changed in order for teams to complete development work.

References

- GitLab Repositories (for guidance on creating a new project)
- Change Management Policy
- GCF Security Control Lifecycle

SECTION: security/standards/password-standard.md

Purpose

This document outlines information security password standards intended to protect GitLab information systems and other resources containing confidential (Red and Orange) GitLab data from unauthorized use, where technically feasible.

Scope

Applies to all GitLab team members, contractors, advisors, and contracted parties interacting with GitLab computing resources and accessing confidential data.

Roles & Responsibilities

Role	Responsibility
-----	-----
GitLab Team Members	Responsible for adhering to the requirements outlined in this standard
Security	Responsible for defining and monitoring implementation of these standards for critical applications
Security Management (Code Owners)	Responsible for approving significant changes and exceptions to these standards

Standard

Constructing secure passwords and ensuring proper password management is essential. GitLab's password standards are based, in part, on the recommendations by NIST 800-63B. To learn what makes a password truly secure, read this article or watch this conference presentation on

password strength.

Note: If a system cannot support a specific configuration in this standard due to technical limitations, the configuration must be set to the closest possible setting that matches this standard. Exceptions are only required when deviating from requirements for business reasons, not technical limitations. For any questions, please reach out to the Security Team.

Password Requirements

- Minimum Length = 12 characters
- Special Characters = No
- Password Reuse = No
- Password expiration = No
- Multi-factor authentication (MFA) = Yes, whenever possible

To make a secure password you can remember, consider using a combination of 5 or more random words. Security questions like "What is your favorite color? What is your mother's maiden name?", etc should be answered with a random non-obvious word or set of words. You can generate answers in 1Password and store them as a note. This helps ensure the answer isn't easily guessable and will be unique across different sites.

Password Management

- Passwords are to be kept private and secured.
- Individual account passwords are not to be shared.
- Passwords are not to be stored in clear text or be written down.
- Password "hints" are not to be used. If a password is forgotten, a mechanism must be in place to replace a password/passphrase with sufficient controls to verify the identity of the requester of the password reset.
- Passwords must be stored in a way that is resistant to offline attacks and must be salted and hashed using a suitable one-way key derivation function.
- If a password is required to be stored, it must be stored within an approved password manager application and may be pasted from this using a master password function (e.g. 1Password).
- If an account or password is suspected to have been compromised, immediately report the incident to Security and promptly follow instructions.

System Password Configuration Requirements

- For systems where a password can be configured the minimum password length needs to be set to 12 characters.
- If a particular system will not support 12 character passwords, then the maximum number of characters allowed by that system shall be used.
- The use of special characters is not required or even recommended.
- If a particular system requires a password history, configuration should be set for 25 remembered passwords.
- Passwords are not acceptable if they match the subsequent patterns, and must be checked against commonly used or expected patterns, including: known breached password lists, dictionary words, repetitive or sequential characters, or context specific words such as the name of the service, username, or derivatives thereof.
- System administrators of applications and/or devices must change default passwords.

- System administrators need to enable password strength on third party applications and/or tools, where applicable.
- For applications where a password is the only source of authentication, a password must be expired within a maximum of 90 calendar days.
- Systems should monitor and log failed login attempts.
- Information related to authentication failed login attempts need to be recorded within the application logs if technically feasible; such as: name, date, number of failed attempts, unique log identifier.
- Repeated failed login attempts need to trigger a temporary account lockout after 10 failed attempts. If the particular system will not support lockout after 10 attempts or less, the lockout needs to be configured to the minimum value allowed by the system. The lockout may end after a designated period of time, or require a manual unlock, depending on the profile of the application.
- Two Factor Authentication (2FA) must be enforced.

Two Factor Authentication (MFA or 2FA)

All GitLab team members are required to use Two Factor Authentication (2FA). Usage of 2FA by GitLab team members is required for access to the production environment. It should be noted that references to MFA (Multi-Factor Authentication) are often included in language associated with third party products and certain compliance references, but the general concept is still covered by the term "2FA". There are different 2FA methods that can be used by GitLab team members. These are ranked by security strength:

- Biometric Authentication. Biometric-based authentication uses "something you are" to authenticate you. Examples include touch-id (fingerprint) or face-id (facial recognition). This is the most secure authentication method.
- WebAuthn. FIDO2/WebAuthn uses a hardware token to authenticate you. It is one of the most secure methods, assuming the hardware token itself is physically secured.
- Push Authentication. For Push Authentication to work, the authentication service and a complementary mobile app typically use RSA keys and OOB (out-of-band) communications to perform the secondary authentication. From a pure cryptographic perspective, this is slightly less secure than WebAuthn as WebAuthn uses secure hardware storage.
- TOTP. TOTP (Time-based One Time Password) is a popular method for a second factor. While not as secure as WebAuthn, Push as TOTP could be phished (although the attack window would be extremely short). It is still a very secure method of authentication. As 1Password is used by GitLab team members, this could be used for TOTP after proper configuration (1Password guidelines).
- SMS. SMS (Short Message Service) is a method of using text messaging to provide out-of-band (OOB) authentication. As the messages can be spoofed or intercepted more easily than other methods, SMS is highly discouraged for 2FA. As of this writing, the Security Department is unaware of GitLab assets or third party applications that team members are using that only support SMS 2FA. If you need to use something that only offers SMS as a second factor for GitLab, contact the Security Department.

For a better understanding of how 2FA fits into GitLab, refer to the Accounts and Passwords section, which includes pointers for setting up passwords, acquiring FIDO2 tokens, and links to further resources. Refer to the Tools and Tips page for more detailed information regarding FIDO2/WebAuthn and other 2FA methods.

Application Authentication Requirements

- Effective FY23 Q3, all third party applications that house GitLab confidential data are required to authenticate via Okta inline with GitLab's approach to centralized authentication and authorization. Security Notices will be required in all cases where Okta is not supported.
- Authentication to an application should contain multi-factor authentication (Token, OTP Generator, SSO, YubiKey).
- SAML Assertion after logging into an authentication portal is required where technically feasible (e.g. Okta).
- Authentication to an application should support individual users, not groups.

Exceptions

Exceptions to this standard will be tracked as per the Information Security Policy Exception Management Process.

References

- Token Management Standard

SECTION: security/standards/physical-security-standard-for-company-assets.md

Purpose

This document defines asset management measures and requirements to support the protection of information assets in GitLab's all remote environment. The measures and requirements noted within the standard are designed to create a secure infrastructure, work environment, and protect sensitive information from physical threats.

Scope

This standard applies to all GitLab team-members, contractors, advisors, and contracted parties interacting with GitLab computing resources and accessing company or customer data.

Roles & Responsibilities

| Role | Responsibility |

|----|-----|

| Security Assurance | Responsible for implementing and executing this standard |

| Security Assurance Management (Code Owners) | Responsible for approving significant changes and exceptions to this standard |

| Team Members, Contractors, Advisors, Contracting Parties | Responsible for adhering to the 'Physical Devices and Location' requirements of this standard |

Overview

As an all remote company, physical protection of information assets can be broken out into a defined "security zone". Security zones are defined as requirements for the handling of

information assets in their physical location.

GitLab has two distinct security zones:

Infrastructure (for SaaS products)

- Hosted and physically secured by third party service provider(s)
- Shared responsibility model
- Adherence to physical security requirements reviewed annually as part of the Third Party Risk Management (TPRM) review and Complementary User Entity Controls (CUEC) review. This includes confirmation that independent third parties attest to effective physical security procedures including but not limited to:

- Visitor Management
- Premises Protection
- Environmental Securities
- Access Management

Physical Devices and Location

- Laptops are protected through Endpoint Management Procedures and secured through system configurations defined in the IT Security - System Configurations handbook page which include, but are not limited to:

- Passwords
- Screen timeout
- Encryption
- Endpoint detection and response

- GitLab implements a Zero Trust architecture that complements physical security measures by requiring authentication and authorization for all resource access, regardless of device location.

- Implement Clear Desk/Clear Screen requirements.

- Ensure devices are not left unattended in public areas and are locked when not in use. Activate a screensaver with password lock, lock the desktop, close the lid.

- Personal mobile phone and tablet usage must be passcode protected.

- Sensitive data should not be stored on removable storage devices, such as USB drives or external hard drives. External storage devices on company assets is not sanctioned.

- Printing documents containing sensitive information as defined by the Data Classification Standard is prohibited.

- Secure your data during travel ensuring that you are in a secure place and no-one can hear you when you are talking about restricted data, and locking your device when it is not in use.

- Do not bring company-owned devices to embargoed countries without consulting the Legal

Department.

Exceptions

Exceptions to this procedure will be tracked as per the Information Security Policy Exception Management Process.

References

- Internal Acceptable Use Policy
- IT Security System Configuration
- Information Security Management System

SECTION: security/standards/records-retention-deletion.md

Purpose

The GitLab records retention and disposal standard lists the specific retention and secure disposal requirements for critical GitLab records. These minimum requirements inform design and maintenance decisions for all GitLab tier 1 and tier 2 critical systems.

Scope

The below retention and secure disposal requirements apply to all GitLab records enumerated in the table below stored in GitLab tier 1 and tier 2 critical systems.

Roles & Responsibilities

Role	Responsibility
-----	-----
GitLab Team Members	Responsible for following the requirements in this controlled document.
Security Compliance Team	Responsible for reviewing and maintaining this controlled document.
Control Owners	Responsible for defining and implementing procedures to support the below requirements.
Security Assurance Management (Code Owners)	Responsible for approving significant changes and exceptions to this controlled document.

Retention & Disposal Requirements Procedure

Record Requirement	Retention Requirement	Disposal
-----	-----	-----

Business continuity plan approvals	3 years	[GCP/AWS
Secure Deletion]		
Business continuity test results	3 years	[GCP/AWS

Secure Deletion]		
Production backup tests	1 year	[GCP/AWS
Secure Deletion]		
Production changes	3 years	[GCP/AWS
Secure Deletion]		
Security policy review/approvals	3 years	[GCP/AWS
Secure Deletion]		
Terms of service acceptance	As long as user is active	[GCP/AWS
Secure Deletion]		
Access request/change records	1 year	[GCP/AWS
Secure Deletion]		
Team-member offboarding issues	Varies by local laws	
[GCP/AWS Secure Deletion]		
System access reviews	1 year 3 months	[GCP/AWS
Secure Deletion]		
Shared and group authentication reviews	1 year 3 months	[GCP/AWS
Secure Deletion]		
Production audit logs	1 year	[GCP/AWS
Secure Deletion]		
GCP firewall configuration artifacts	1 year	[GCP/AWS
Secure Deletion]		
Job roles and responsibilities	1 year	[GCP/AWS
Secure Deletion]		
Security incident communication to customers	3 years	[GCP/AWS
Secure Deletion]		
Personnel-file records	Varies by local laws	[GCP/AWS
Secure Deletion]		
1:1 meeting notes	Varies by local laws	[GCP/AWS
Secure Deletion]		
On-boarding tickets	Varies by local laws	
[GCP/AWS Secure Deletion]		
Annual risk assessment report	2 years	[GCP/AWS
Secure Deletion]		
Risk treatment plans	3 years	[GCP/AWS
Secure Deletion]		
Security observation issues	3 years	[GCP/AWS
Secure Deletion]		
Board of Directors meeting minutes	Indefinite	N/A
Release notes	1 year	[GCP/AWS
Secure Deletion]		
Critical system activity logs	60 days	[GCP/AWS
Secure Deletion]		
Security monitoring alerts/metrics	3 years	[GCP/AWS
Secure Deletion]		
Vendor security review issues	3 years	[GCP/AWS
Secure Deletion]		
Customer-signed MSA's	Indefinite	N/A
Vendor NDA's	Indefinite	N/A

Annual security awareness training records	3 years	[GCP/AWS
Secure Deletion]		
Secure coding training records	2 years	[GCP/AWS
Secure Deletion]		
Penetration testing reports and remediation issues	2 years	[GCP/AWS
Secure Deletion]		
System patch records	1 year	[GCP/AWS
Secure Deletion]		
Source code scanning results	1 year	[GCP/AWS
Secure Deletion]		
ZenDesk tickets	3 years	[GCP/AWS
Secure Deletion]		
Nonpublic information review records	3 years	[GCP/AWS
Secure Deletion]		
Role-based security training records	3 years	[GCP/AWS
Secure Deletion]		
Audit log review records	3 years	[GCP/AWS
Secure Deletion]		
Security assessment reports/observation	3 years	[GCP/AWS
Secure Deletion]		
Security configuration reviews/alerts	3 years	[GCP/AWS
Secure Deletion]		
Security authorization records	3 years	[GCP/AWS
Secure Deletion]		
System connection requirements	3 years	[GCP/AWS
Secure Deletion]		
Configuration change records	3 years	[GCP/AWS
Secure Deletion]		
Security impact analysis records	3 years	[GCP/AWS
Secure Deletion]		
Production asset inventory	3 years	[GCP/AWS
Secure Deletion]		
BC training records	3 years	[GCP/AWS
Secure Deletion]		
Production backups	Organizationally-defined	[GCP/AWS
Secure Deletion]		
Customer data backups	Organizationally-defined	[GCP/AWS
Secure Deletion]		
Employment applications and interview notes (US-based applicants only)	4 years (updated	
2021-07) N/A		
Temporary Files with PII data	As long as needed for business purpose	
Per System's default deletion schedule		

Exceptions

Exceptions to these requirements will be tracked as per the Information Security Policy Exception Management Process.

References

- Controlled Document Procedure
- Data Classification Standard
- Google Cloud Platform data deletion

SECTION: security/standards/security-logging-standard.md

Purpose and Scope

The purpose of this security logging standard is to define GitLab's requirements for security logging. This document covers both security logging in GitLab's SIEM (Devo) as well as security logging requirements for systems not sending logs to GitLab's SIEM.

Roles & Responsibilities

Role	Responsibility
GitLab System Owners (as defined in GitLab's tech stack)	Directly Responsible for adhering to the requirements outlined in this standard
Security Operations Team	Directly Responsible for the prioritization, onboarding and maintenance of logs in the Security team's SIEM (Security Information Event Management) system, Devo.

Security Logging Requirements

Security logs are generated by applications and systems used on GitLab and are primarily used for security monitoring, security incident response and cyber threat hunting.

The GitLab Security Operations team iteratively evaluates the priority of security logs for existing systems, and evaluates requests for new log sources that must be onboarded for improved security observability. The Security Operations team maintains a Single Source of Truth (SSOT) for the security logs in their SIEM.

As new applications, systems and services are onboarded at GitLab, if a system's security logs (e.g. App, OS, transaction logs) are not sent to GitLab's SIEM, security logs must be stored in approved secure storage for a duration aligned with our retention policy.

Security Log Retention Requirements

Security Log Description	Retention Requirements
Security logs for production systems	1 year minimum
Security logs for critical systems that are not production	90 days minimum

Security Log Collection Requirements

At a minimum, authentication and transaction events must be collected for GitLab production systems and also for non-production critical GitLab systems.

A detailed description of security events and fields is outlined below.

Security Log Event Type	Required?	Security Log Field
Authentication Events	Yes	User
Authentication Events	Yes	Timestamp
Authentication Events	Yes	Source IP address
Authentication Events	Yes	Destination IP address
Authentication Events	Yes	Action that was attempted
Authentication Events	Yes	Action result (success or failure)
Transaction Events	Yes	User
Transaction Events	Yes	Timestamp
Transaction Events	Yes	Source IP address
Transaction Events	Yes	Action that was attempted
Transaction Events	Yes	Action result (success or failure)
Data Access Events	No - Dependent on log verbosity	User
Data Access Events	No - Dependent on log verbosity	Timestamp
Data Access Events	No - Dependent on log verbosity	Source IP address
Data Access Events	No - Dependent on log verbosity	Action that was attempted
Data Access Events	No - Dependent on log verbosity	Accessed resource
Data Access Events	No - Dependent on log verbosity	Action result (success or failure)
Alert Events	No - Dependent on log verbosity	Alert name
Alert Events	No - Dependent on log verbosity	Timestamp
Alert Events	No - Dependent on log verbosity	User
Alert Events	No - Dependent on log verbosity	Source IP address
Security Alert Events	No - Dependent on log verbosity	Action
Security Alert Events	No - Dependent on log verbosity	Additional alert details

SECTION: security/standards/software-development-lifecycle-standard.md

Purpose

Secure software development is critical to developing and maintaining a safe and trusted application. This standard outlines the general components of GitLab's software development lifecycle.

Scope

This standard applies to anyone developing code at GitLab in support of GitLab's production applications. For in depth view of development process, see product development flow.

Roles and responsibilities

Role	Responsibility
Security Governance	Responsible for creating and implementing this standard
Team members	Responsible for execution of the standard statements

Standard

Inception and Requirements

This stage occurs across different mediums depending upon each team's individual processes.

During this stage, the following information is established:

- problem statement and desired results
- scope is defined
- identification of key stakeholders
- a detailed project plan with milestones and deliverables is created in coordination with relevant stakeholders

At a minimum, requirements must identify:

- what the application or functionality will do
- the resources required to complete the project

Identified requirements are documented in the project management tool and made available for review and approval by relevant stakeholders.

Design

During the design stage, design documents are captured in the project management tool as version controlled documents.

Considerations for design documents are:

- Architecture: Teams define if they want a specific type of template or if they want to implement any type of industry practice.
- User Interface: Teams define the way users will be interacting with the application or function.
- Security: Developers must define how they will keep the application secure. This includes deciding how to protect user data and general application data.
- Programming: Define the project's tech and tool stacks.
- Components: Define the components that will be needed to support the solution.

Design documents must be approved by relevant stakeholders prior to being merged (prototyping).

Development and Testing

Software development at GitLab is accomplished using industry standard development practices.

Software development must be accomplished using GitLab's development platform in a version controlled manner.

Development must adhere to Code review guidelines and secure coding best practices (to avoid introducing vulnerabilities).

Development work must undergo testing and approval prior to deployment to production. This includes configuration changes and other related changes that may not be considered "development" work.

Deployment

Deployment of software at GitLab must adhere to all defined staging, testing, release, and rollback processes. Details on GitLab's deployment process can be found [here](#)

After any necessary verification in production is completed, details of changes are pushed to the releases page to be communicated as part of the relevant release notes.

Maintenance

GitLab continuously monitors the state and stability of its platforms.

Maintenance of GitLab's platform is conducted in accordance with GitLab's release and maintenance policy.

SECTION: security/standards/token-management-standard.md

Purpose

The Token Management Standard defines approved GitLab token usage, settings, and distribution for the purposes of providing authentication and authorization within the various systems and subsystems used by the GitLab product.

For some elements of this standard, there will be technology, techniques, settings, and variations thereof being referenced that are currently not implemented. In those cases, this standard is meant to provide specific guidance for future development and implementation.

The Token Management Standard allows for a more consistent approach to token usage within GitLab, easier adaptation to industry standards and compliance frameworks (such as FedRAMP), and an overall more secure product and working environment. In addition, most of the standards are based off recommendations from NIST (National Institute of Standards and Technology) as many of the compliance frameworks are based on NIST standards, and NIST consistently makes solid recommendations that are adopted by many organizations globally.

Scope

The Token Management Standard applies to all GitLab team members, contractors, consultants, vendors, and other service providers that handle, manage, store or transmit GitLab data.

This is required for coding best practices as well as account and authenticator management involving tokens for the GitLab product itself. Basically, if it touches GitLab or GitLab customer data, the standard applies.

Roles & Responsibilities

Role	Responsibility
GitLab Team Members	Responsible for adhering to the requirements outlined in this standard
Security Management and Cryptographic Officer	Responsible for approving changes and exceptions to this standard
DRI for Token(s)	Reviews and approval authority for all Owner, and Maintainer role assignments
Management	Review and approval authority for all Group and Project memberships
Group Owners, Group Maintainers, Project Owners, Project Maintainers	Group and Project account management, and token management duties including but not limited to creating, revoking, and monitoring the use and distribution of Group, Project, Deploy, and Runner Token objects

GitLab Responsibilities

GitLab team members, contractors, consultants, vendors and other service providers are required to regularly review and understand this Token Management Standard, and how to request, approve, distribute, use and protect GitLab tokens to maintain security. This document is subject to be updated at any time, the aforementioned people are responsible for tracking changes.

Customer Responsibilities

GitLab customers are responsible for managing their own accounts and tokens, should consider these standards as highly recommended, and adopt them according to their own internal requirements. GitLab handles Customer Data internally according to our non-disclosure obligations written in our Mutual Non-Disclosure Agreement, and we use the controls identified in this standard.

Standard

Compliance and Certification Standards

General Assumptions

- The standard is currently not fully implemented within the GitLab product, but will help guide future development to achieve the standard.
- The standard meets all relevant compliance needs (e.g. FedRAMP requirements).
- Personal Access Tokens (PATs) will be highly discouraged within the GitLab production environment, and disallowed/disabled wherever possible. Existing tokens shall remain, but additional issuance will not be permissible/possible.
- Many of the tokens can be disabled, and within certain environments (e.g. FedRAMP) they will have to be unless they can meet creation, usage, storage and access standards as defined below.
- The main facilitator of the data being created and used within the GitLab environment will be referred to as the Site Administrator .

Token and Account Management

1. Due to the nature of the GitLab application, and the ability of a malicious actor that has access to a GitLab instance to cause damage to information contained in the instance, account

type definitions are necessary. A GitLab instance has both privileged and non-privileged accounts within the application. This privilege level